
CADFit: Precise Mesh-to-CAD Program Generation with Hybrid Optimization

Ghadi Nehme¹ Eamon Whalen² Faez Ahmed¹

Abstract

Despite recent progress, recovering parametric CAD construction sequences from geometric input, such as meshes or point clouds, is a key challenge for design and manufacturing, as existing CAD reconstruction and generation methods are largely restricted to difficult-to-edit formats like meshes or Breps or editable simple sketch-and-extrude pipelines and low-complexity datasets. We introduce **CADFit**, a hybrid optimization-based CAD reconstruction framework that recovers complex, editable CAD construction sequences from meshes by incrementally fitting and validating parametric operations using geometric feedback. Our approach is distinguished by formulating reconstruction as an IoU-driven optimization over structured CAD programs and supporting a rich set of operations, including extrusions, revolutions, fillets, and chamfers. Experiments on multiple CAD benchmarks show that CADFit outperforms state-of-the-art mesh-to-CAD methods in volumetric Intersection-over-Union and Chamfer Distance, while substantially reducing the Invalid Ratio of reconstructed CAD programs, particularly for complex designs. We further present a multimodal pipeline that enables end-to-end reconstruction of CAD construction sequences from images by combining image-based geometry reconstruction with CADFit. By enabling accurate reconstruction of higher-complexity CAD models, CADFit provides a practical foundation for generating richer datasets and advancing future learning-based approaches to CAD reverse engineering. The code is available at: <https://github.com/ghadinehme/CADFit>.

¹Department of Mechanical Engineering, Massachusetts Institute of Technology, Cambridge, MA 02139, USA ²Siemens Digital Industries Software, Plano, TX 75024, USA. Correspondence to: Ghadi Nehme <ghadi@mit.edu>.

1. Introduction

Parametric CAD models represent geometry as structured construction sequences composed of geometric operations with continuous parameters. Such representations are widely used in engineering workflows because they enable compact shape specification and support downstream editing and constraint-based modeling (Camba et al., 2016; Li et al., 2010). Recovering parametric construction sequences from raw geometric data, however, remains a challenging inverse problem. A single target shape may admit many distinct but geometrically valid construction sequences (Lam-bourne et al., 2022; Willis et al., 2021), while each sequence must satisfy strict geometric constraints. Moreover, the problem requires jointly inferring discrete operation choices (e.g., feature type and ordering) and continuous parameters (e.g., dimensions and placements), making the search space highly non-convex and combinatorial.

Despite recent progress in CAD generation and reconstruction, existing AI-based methods remain limited in both operational expressiveness and geometric complexity. Most current CAD generation models focus on a narrow subset of operations, typically sketch-and-extrude pipelines, and are trained on datasets containing relatively simple shapes with few interacting features (Khan et al., 2024; Alam & Ahmed, 2024; Wang et al., 2025). As a consequence, these models struggle to represent realistic CAD designs involving multiple construction operations, complex boolean logic, and finishing operations such as fillets and chamfers. This limitation is not only a modeling issue but also a data bottleneck: the lack of accurate tools for reconstructing complex CAD construction sequences has constrained the scale and diversity of datasets available for learning.

In this paper, we introduce **CADFit**, a hybrid optimization-based method for reconstructing CAD construction sequences directly from geometry. Given a watertight input mesh, CADFit recovers an ordered program composed of parametric operations drawn from a rich operator set, including extrusions, revolutions, fillets, and chamfers, and composed using Boolean union and cut. Rather than predicting programs in a single forward pass, CADFit formulates reconstruction as an explicit optimization problem over executable CAD programs, guided by geometric feedback.

The core insight behind CADFit is that valid CAD programs

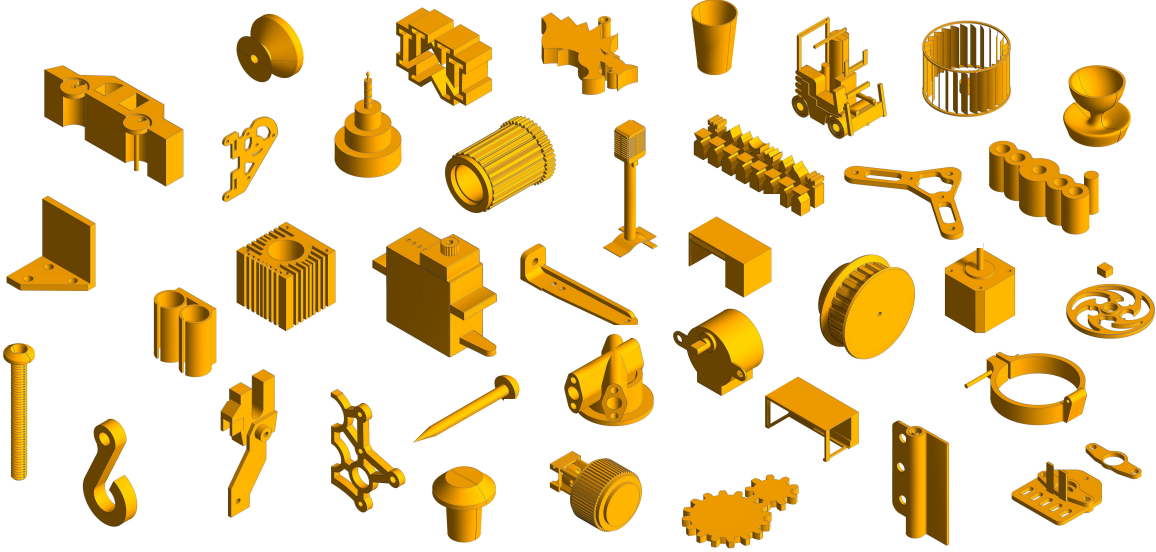


Figure 1. Mesh-to-CAD construction sequences generated by CADFit with varying complexity, including extrusions, revolutions, fillets, and chamfers.

can be recovered by tightly coupling geometric reasoning with program execution. CADFit first extracts candidate sketch profiles from the input mesh, then generates a small set of geometrically consistent operation candidates by analyzing how parametric sweeps affect surface alignment. These candidates are assembled into a construction sequence using an IoU-guided selection procedure that explicitly executes and validates each operation. To reduce the combinatorial search space, CADFit additionally trains a sketch selection model that predicts which extracted sketch profiles are likely to contribute to the final construction sequence. This learned prior is trained using labels automatically obtained from optimization-based reconstructions and is used only to filter candidates, without affecting correctness.

Importantly, CADFit decouples geometric perception from parametric reconstruction. The method operates on surface geometry and can therefore be composed with upstream models that reconstruct meshes from images or point clouds. This design enables a unified pipeline for mesh-, point-cloud-, and image-based CAD reconstruction without re-training or modifying the core algorithm.

We evaluate CADFit on multiple benchmarks, including DeepCAD (Wu et al., 2021), Fusion360 Gallery (Willis et al., 2021), and ABC (Koch et al., 2019), spanning a wide range of geometric complexity. Across all benchmarks, CADFit achieves state-of-the-art performance in volumetric IoU and Chamfer Distance while maintaining zero invalid outputs. Beyond geometric accuracy, CADFit recovers compact construction sequences that are often more concise than those found in existing CAD datasets, suggesting improved supervision for future learning-based approaches.

Contributions. We make the following contributions:

- **Hybrid optimization for executable mesh-to-CAD programs.** We propose CADFIT, an optimization-based framework that reconstructs *executable* Cad-Query/OpenCascade CAD construction sequences from a watertight input mesh by explicitly optimizing volumetric IoU under CAD-kernel validation, supporting extrude, revolve primitives, Boolean (union & cut), and finishing operations (fillet & chamfer).
- **Geometry-driven candidate generation and compact program search.** We introduce a reconstruction pipeline that (i) extracts sketch profiles from planar face clusters and slicing planes, (ii) generates a small set of high-quality extrude & revolve candidates via one-sided Chamfer parameter sweeps and stable-interval selection, (iii) assembles compact sequences via IoU-guided backward pruning and iterative residual reconstruction with union & cut, and (iv) accelerates search using a learned sketch prior trained from optimization-derived supervision.
- **Complexity-aware benchmarks and multimodal reconstruction.** We establish and evaluate on benchmarks spanning a wide range of CAD complexity (DeepCAD, Fusion360, and ABC stratified by complexity), and demonstrate consistent gains in IoU/Chamfer and near-zero invalid outputs for both mesh-to-CAD and an end-to-end image-to-CAD pipeline (image to mesh to CADFit).

2. Related Work

Learning-Based CAD Program Generation. There has been increasing interest in neural methods that generate or reconstruct CAD models as sequences of parametric operations. Early approaches such as DeepCAD (Wu et al., 2021) introduce a domain-specific language (DSL) for sketch-and-extrude programs and learn sequence autoencoders over this representation. Subsequent methods extend this paradigm to richer inputs and outputs, including point clouds and meshes, and generate executable CAD code using structured decoders or code-generation models (Xu et al., 2022; Ren et al., 2022; Li et al., 2023; Yu et al., 2025; Rukhovich et al., 2025; Li et al., 2025). While these methods produce editable outputs, they rely on amortized sequence prediction and often struggle with long, compositional programs, complex boolean interactions, and finishing operations.

Recent work further explores multimodal conditioning and large language models for CAD code generation (Doris et al., 2025; Kolodiazhyi et al., 2025; Alrashedy et al., 2025). These approaches broaden the range of inputs and tasks but inherit the brittleness of autoregressive program generation and do not explicitly optimize for geometric fidelity or program optimality. In contrast, CADFit avoids direct sequence prediction and instead reconstructs construction programs through geometry-driven optimization.

Geometry-Driven CAD Reverse Engineering and B-rep Reconstruction. Another line of work focuses on recovering CAD structure from geometric input using reverse engineering techniques. Classical pipelines decompose shapes into primitives and fit analytic surfaces, but struggle to assemble consistent parametric models for complex designs. Recent learning-assisted methods improve robustness by combining neural segmentation with analytic fitting, particularly for boundary representation (B-rep) reconstruction (Jayaraman et al., 2022; Liu et al., 2024; Xu et al., 2024; Lee et al., 2025). These methods produce accurate surface topology and geometry, but do not recover ordered construction sequences or design intent encoded in parametric CAD programs.

Inverse CSG methods formulate reconstruction as a combinatorial or optimization-based search over boolean programs (Du et al., 2018; Kania et al., 2020). Although such formulations improve validity guarantees, they typically operate over simple primitive sets and do not scale to sketch-based operations, revolutions, or finishing features. CADFit differs by optimizing directly over parametric CAD operations while maintaining explicit geometric validation.

Structured CAD Representations and Datasets. Large-scale datasets such as ABC, DeepCAD, and Fusion360 have enabled learning structured representations of CAD models, including sketches, feature trees, and operation

graphs (Koch et al., 2019; Wu et al., 2021; Willis et al., 2021; Xu et al., 2023; Uy et al., 2022; Man et al., 2026; Liu et al., 2024). More recent work studies neural representations of sketches, loops, and construction graphs to improve interpretability and controllability (Xu et al., 2023; Karadeniz et al., 2025). However, construction sequences in these datasets are often redundant or suboptimal, reflecting human modeling practices rather than canonical programs.

CADFit complements these efforts by reconstructing shorter, cleaner construction sequences through explicit optimization, which can serve as higher-quality supervision for future learning-based models.

Multimodal 3D Reconstruction Pipelines. Advances in image- and point-based 3D reconstruction have enabled pipelines that combine perception models with downstream CAD inference. Methods such as Pixel2Mesh (Wang et al., 2018), Trellis (Xiang et al., 2025), and Hunyuan3D (Zhao et al., 2025) recover high-quality surface geometry from images or sparse observations. Several recent works combine such models with CAD reconstruction (Li et al., 2025), but often require modality-specific adaptations or retraining.

CADFit adopts a decoupled design in which perception models recover surface geometry, while parametric reconstruction is performed via geometry-driven optimization, enabling robust CAD program recovery even from remeshed and smoothed reconstructed surfaces.

3. Method

We introduce **CADFit**, a hybrid optimization-based framework for recovering editable CAD construction programs from geometric input. Given a watertight mesh $\mathcal{M} \subset \mathbb{R}^3$, CADFit reconstructs an ordered CAD program $\Pi = (o_1, \dots, o_T)$ whose execution produces a solid closely approximating $\mathcal{M}$. Each operation o_t belongs to a fixed parametric operator set $\mathcal{O} = \{\text{EXTRUDE}, \text{REVOLVE}, \text{FILLET}, \text{CHAMFER}, \text{UNION}, \text{CUT}\}$. The complete procedure is summarized in Fig. 2.

Problem Formulation. Let $\text{Solid}(\Pi)$ denote the solid obtained by executing program Π . We seek a valid program maximizing volumetric overlap with the target mesh:

$$\Pi^* = \arg \max_{\Pi \in \mathcal{S}} \text{IoU}(\text{Solid}(\Pi), \mathcal{M}), \quad (1)$$

where $\mathcal{S}$ denotes the space of syntactically and geometrically valid CAD programs. Boolean UNION and CUT operations are applied sequentially to compose intermediate solids.

CAD Program Representation. Each operation $o_t \in \Pi$ is represented as a tuple $o_t = (\tau_t, \theta_t, R_t)$, where τ_t is the operation type, $\tau_t \in \{\text{EXTRUDE}, \text{REVOLVE}, \text{FILLET}, \text{CHAMFER}, \text{UNION}, \text{CUT}\}$, θ_t denotes the continuous operation parameters (e.g., extru-

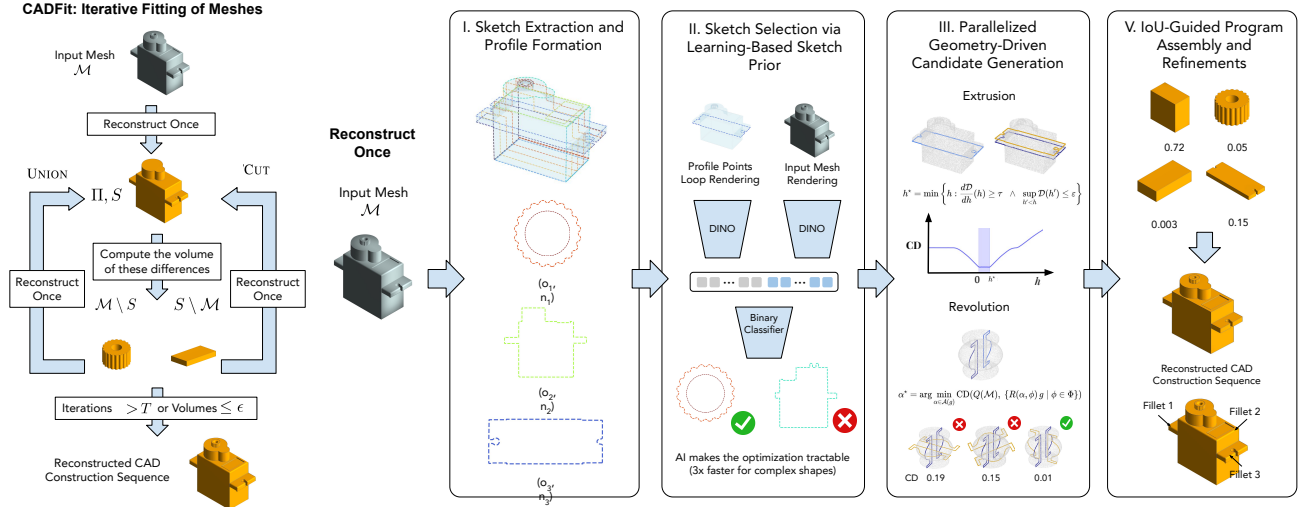


Figure 2. Overview of the CADFit Pipeline for Iterative Mesh-to-CAD Reconstruction

sion height, revolution angle and axis, fillet radius, chamfer length), and R_t specifies the geometric references required by the operation. Depending on τ_t , R_t may refer to a sketch profile, a set of edges, or one or more existing solids.

We implement CADFit using **CadQuery** (contributors, 2026), an OpenCascade-based parametric CAD library. CAD programs Π are represented directly using the CadQuery operation grammar, including `extrude`, `revolve`, `fillet`, `chamfer`, `union`, and `cut`, with sketches defined on explicit planes using line, arc, and circle primitives (Fig. 3). Kernel feedback is used to reject invalid operations; operations that fail to execute are discarded.

Pipeline Overview. CADFit reconstructs a CAD program through iterative mesh fitting. The method consists of an outer residual-reconstruction loop and an inner single-pass reconstruction procedure. Given an input mesh $\mathcal{M}$, CADFit first runs a single-pass reconstruction to obtain an initial CAD program Π_0 and reconstructed solid S_0 . It then computes two complementary residuals: the under-reconstructed region $R_k^+ = \mathcal{M} \setminus S_k$, which contains target geometry missing from the current reconstruction, and the over-reconstructed region $R_k^- = S_k \setminus \mathcal{M}$, which contains geometry present in the reconstruction but absent from the target. CADFit applies the same single-pass reconstruction procedure to these residual meshes and combines the recovered residual programs with the current program using Boolean operations: under-reconstructed residuals are added with `UNION`, while over-reconstructed residuals are removed with `CUT`. This process can be repeated for a bounded number of iterations, progressively refining the CAD program.

The inner single-pass reconstruction procedure has four stages. First, CADFit extracts candidate sketch profiles from

the input mesh using planar face clusters and axis-aligned slicing planes. Second, a learned sketch prior filters the extracted profiles to reduce the number of noisy or irrelevant candidates before expensive CAD-kernel evaluation. Third, CADFit performs geometry-driven candidate generation by fitting `EXTRUDE` and `REVOLVE` operations to the retained sketch profiles through parameter sweeps. Finally, CADFit assembles a compact program in two stages: it first greedily selects candidates that improve IoU with the target mesh, and then applies IoU-guided backward pruning to remove candidates whose deletion does not reduce reconstruction quality. This nested design allows CADFit to recover complex shapes by repeatedly fitting simpler residual geometries while keeping each single-pass reconstruction tractable and compact. (Figure 2)

Single-Pass Reconstruction. The single-pass reconstruction procedure is the basic fitting primitive used both for the initial mesh and for residual meshes in later iterations. Given a target mesh, it extracts sketch profiles, filters them with the learned sketch prior, generates parametric `EXTRUDE` and `REVOLVE` candidates, and assembles a compact program by greedily selecting IoU-improving candidates followed by IoU-guided backward pruning.

Sketch Extraction and Profile Formation. CADFit constructs a finite set of sketch profiles $\mathcal{G} = \{g_i\}$ from $\mathcal{M}$ using explicit geometric operators. Candidate sketch planes are defined by (i) detected planar face clusters and (ii) a fixed set of axis-aligned slicing planes. For each plane $(\mathbf{o}, \mathbf{n})$, we compute local mesh sections and project resulting intersection curves onto the plane, yielding ordered 2D loops. Degenerate loops are discarded, and remaining loops are grouped by polygon containment to form sketch profiles with outer boundaries and interior holes. All geometric extraction steps are detailed in Appendix I.

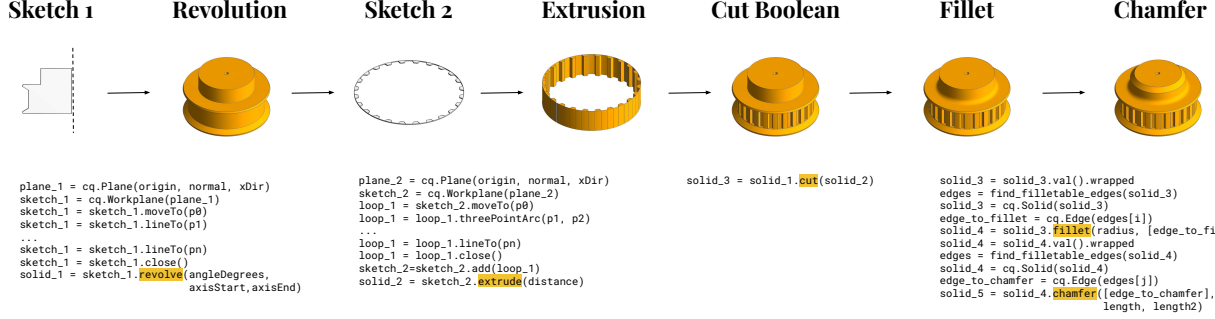


Figure 3. Executable CadQuery program reconstructed by CADFit, illustrating sketch-based primitives, extrusion and revolution operations, Boolean cut and union, and subsequent fillet and chamfer features.

Learning-Based Sketch Prior. To keep the sketch profile set $\mathcal{G}$ tractable, CADFit uses a learned prior $f(\mathcal{M}, g) \in [0, 1]$, which estimates whether a candidate profile g is useful for reconstructing mesh $\mathcal{M}$.

The prior is trained from CADFit’s pure optimization pipeline. Profiles used in the recovered CAD program are labeled positive, while the remaining extracted profiles are labeled negative. Instead of encoding the mesh and sketch separately, we form a joint visual input: CADFit renders multi-view images of $\mathcal{M}$ and overlays the candidate loop g on each view. These images are passed through a DINO encoder (Oquab et al., 2024) to obtain a global visual embedding, which is then fed to a lightweight 3-layer MLP with ReLU activations and a sigmoid output. The resulting score $p(g | \mathcal{M}) = f(\mathcal{M}, g)$ measures mesh–sketch compatibility.

At inference time, each profile is independently selected with probability $p(g | \mathcal{M})$. This converts the prior into a stochastic filter: it favors sketches likely to appear in the final CAD program while preserving exploration. The selected profiles are then fit with sketch primitives. Additional details are provided in Appendix M.

Geometry-Driven Candidate Generation. For each sketch profile $g \in \mathcal{G}$, CADFit generates a small discrete set of EXTRUDE and REVOLVE candidates by solving a continuous parameter selection problem. Let $Q(\mathcal{M})$ denote a surface point cloud sampled from $\mathcal{M}$. For a candidate operation $o(g, \theta)$, we define a one-sided Chamfer objective

$$\mathcal{D}(\theta) = \frac{1}{|P(g, \theta)|} \sum_{\mathbf{x} \in P(g, \theta)} \min_{\mathbf{y} \in Q(\mathcal{M})} \|\mathbf{x} - \mathbf{y}\|_2^2,$$

where $P(g, \theta)$ are surface points induced by applying the operation with parameters θ to profile g . Rather than minimizing $\mathcal{D}$ directly, we sweep θ over a bounded domain and identify stable parameter intervals preceding sharp increases in $\mathcal{D}$. These intervals correspond to geometrically consistent operations that do not overshoot the target shape. All parameter selection details are provided in Appendix K.

IoU-Guided Program Assembly. Let $\mathcal{C} = \{c_j\}$ denote the set of all generated operation candidates. CADFit assembles a base program using a forward greedy selection stage followed by backward pruning. Starting from an empty program $\Pi = \emptyset$ and solid $S = \emptyset$, CADFit repeatedly adds the candidate that gives the largest positive IoU improvement:

$$c^* = \arg \max_{c \in \mathcal{C} \setminus \Pi} [\text{IoU}(S \cup c, \mathcal{M}) - \text{IoU}(S, \mathcal{M})].$$

If this improvement is positive, c^* is added to Π and merged into S . Forward selection terminates when no remaining candidate improves IoU. After this forward selection stage, CADFit applies backward marginal pruning to remove redundant candidates. Starting from the greedily selected program, candidates are iteratively removed according to their marginal removal effect

$$\Delta^-(c | S) = \text{IoU}(\text{Solid}(\Pi \setminus \{c\}), \mathcal{M}) - \text{IoU}(S, \mathcal{M}),$$

removing at each step the candidate with the largest Δ^- . Pruning stops when removing any remaining candidate would decrease IoU. The resulting subset is compact and contains only candidates that contribute to reconstruction quality under union composition. The complete forward-selection and backward-pruning procedure is provided in Algorithm 7 in Appendix L.

Iterative Residual Reconstruction. After the initial single-pass reconstruction, CADFit refines the program by repeatedly fitting residuals between the current reconstruction and the target mesh. At iteration k , the under-reconstructed residual $R_k^+ = \mathcal{M} \setminus S_k$ captures missing target geometry, while the over-reconstructed residual $R_k^- = S_k \setminus \mathcal{M}$ captures excess reconstructed geometry. Each residual is reconstructed using the same single-pass procedure: the program recovered from R_k^+ is added to the current reconstruction with UNION, while the program recovered from R_k^- is subtracted with CUT. Iterations terminate when residual volumes fall below a threshold ϵ or when a fixed iteration budget is reached. The full procedure is given in Algorithm 1.

Algorithm 1 Iterative Residual CAD Program Fitting

Input: mesh $\mathcal{M}$, tolerance ϵ , max iterations T
Output: CAD program Π^*
 $(\Pi, S) \leftarrow \text{RECONSTRUCTONCE}(\mathcal{M})$
for $t = 1$ **to** T **do**
 $R^+ \leftarrow \mathcal{M} \setminus S, R^- \leftarrow S \setminus \mathcal{M}$
 if $\text{Vol}(R^+) \leq \epsilon$ **and** $\text{Vol}(R^-) \leq \epsilon$ **then**
 break
 end if
 Reconstruct R^+ and R^- and combine via UNION/CUT
end for
return Π^*

Application to Multimodal CAD Reconstruction. CADFit is formulated as a mesh-to-CAD reconstruction method but is agnostic to the source of the mesh. By treating geometry as an intermediate representation, CADFit can be composed with upstream perception models that reconstruct surface geometry from images or point clouds. For image-based inputs, we obtain a surface mesh using a pretrained image-to-3D model and apply standard post-processing to satisfy CADFit’s geometric assumptions. The processed mesh is then passed directly to CADFit without modification of the reconstruction pipeline. Implementation details are provided in Appendices C and D. Importantly, CADFit is not retrained or adapted for different input modalities. The same optimization procedure is applied unchanged to meshes originating from images, point clouds, or CAD exports, enabling a clean separation between perception and parametric reconstruction. We evaluate the resulting end-to-end pipeline in Section 4.

4. Experiments

We evaluate CADFit on two settings: **mesh-to-CAD** and **image-to-CAD**. Across both settings, the goal is to recover an editable CAD construction program whose execution yields a solid that matches the target.

4.1. Tasks and Baselines

Mesh-to-CAD. Given a watertight input mesh $\mathcal{M}$, methods must output a CAD construction sequence (or an equivalent executable program) that reconstructs $\mathcal{M}$. We compare against three baselines: GenCAD-3D (Yu et al., 2025), CAD-Recode (Rukhovich et al., 2025), and Cadrille (Koldiazhnyi et al., 2025). When a baseline does not support mesh input, we follow its standard preprocessing and provide point-cloud input.

Image-to-CAD. Given a single RGB image, we first reconstruct an intermediate surface mesh using a pretrained image-to-3D model. We adopt Hunyuan3D (Zhao et al., 2025) after comparing it against another state-of-the-art

image-to-mesh method, Trellis (Xiang et al., 2025), with an ablation study reported in Appendix 8. The reconstructed mesh is then post-processed to satisfy CADFit’s geometric assumptions by enforcing watertightness, applying Taubin smoothing (Taubin, 1995) to suppress high-frequency artifacts, and performing mesh decimation (Garland & Heckbert, 1997). All mesh-to-CAD baselines (GenCAD-3D, CAD-Recode, Cadrille) are run on these processed meshes. We additionally compare against a direct image-to-CAD program generation baseline CAD-Coder (Doris et al., 2025).

4.2. Benchmarks

We construct benchmarks spanning a wide range of geometric complexity. Throughout, we use *STEP face count* as a proxy for CAD complexity (a common and effective complexity measure (Contero et al., 2023)).

DeepCAD (complex subset). We evaluate on the official DeepCAD test split and select the most complex CAD models according to STEP face count to avoid over-representing trivial primitives (e.g., cylinders and boxes). The resulting DeepCAD test set contains 300 shapes.

Fusion360 Gallery (uniform face-count). We sample 300 CAD models from Fusion360 Gallery such that the STEP face count is approximately uniform over a predefined range. This benchmark stresses reconstruction across diverse geometries while controlling for complexity distribution.

ABC (stratified by complexity). ABC contains shapes spanning from very simple to extremely complex CAD. We partition ABC into three complexity windows using STEP face count: Easy: [1, 15], Medium: [16, 150], Hard: [151, 1500], and sample 300 shapes uniformly over face count within each window. Shapes with more than 1500 faces are excluded to avoid extreme outliers.

ABC Image-to-CAD benchmark. For image-to-CAD, we sample 150 shapes from each ABC window (Easy/Medium/Hard), render an isometric view per shape using Blender, and use these images as inputs (Fig. 9).

4.3. Evaluation Metrics

We evaluate using Chamfer Distance (CD), volumetric Intersection-over-Union (IoU), and Invalid Ratio (IR). Predictions are rigidly aligned with a global scale to ground truth by minimizing symmetric CD prior to computing CD and IoU. CD is the symmetric Chamfer Distance between surface point clouds, with ground-truth meshes normalized to $[-1, 1]^3$. IoU is the Intersection over Union between the generated and ground truth solids: $\text{IoU}(A, B) = \text{Vol}(A \cap B) / \text{Vol}(A \cup B)$. IR measures the fraction of invalid outputs (e.g., failed CAD program execution). Details are in Appendix B.

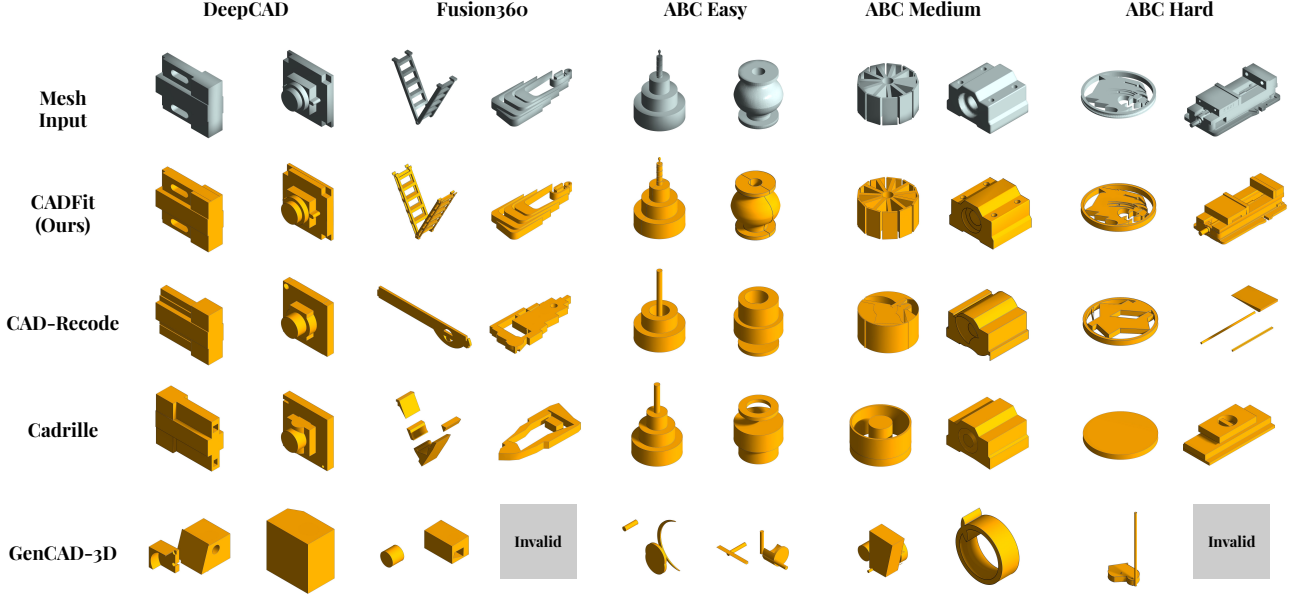


Figure 4. Qualitative mesh-to-CAD reconstruction results on DeepCAD, Fusion360, and ABC (easy, medium and hard subsets).

Table 1. CAD construction sequence reconstruction results on DeepCAD and Fusion360, and on ABC stratified by CAD complexity. Improvement is computed relative to the strongest baseline in each column (max IoU / min CD / min IR).

Method	DeepCAD			Fusion360			ABC								
	IoU↑	CD↓	IR↓	IoU↑	CD↓	IR↓	Easy			Medium			Hard		
							IoU↑	CD↓	IR↓	IoU↑	CD↓	IR↓	IoU↑	CD↓	IR↓
GenCAD-3D	0.255	0.132	0.020	0.208	0.143	0.313	0.258	0.141	0.263	0.204	0.135	0.377	0.135	0.128	0.390
CAD-Recode	0.818	0.027	0.000	0.761	0.027	0.003	0.773	0.030	0.003	0.571	0.044	0.020	0.333	0.054	0.010
Cadrille	0.872	0.020	0.007	0.817	0.019	0.003	0.808	0.028	0.003	0.653	0.032	0.017	0.438	0.040	0.013
CADFit (Ours)	0.964	0.016	0.000	0.921	0.017	0.000	0.908	0.022	0.000	0.851	0.026	0.000	0.662	0.032	0.000
Improvement (%)	↑ 10.6	↓ 19.3	0.0	↑ 12.7	↓ 10.2	↓ 100.0	↑ 12.4	↓ 21.4	↓ 100.0	↑ 30.3	↓ 18.8	↓ 100.0	↑ 51.1	↓ 20.0	↓ 100.0

4.4. Results and Discussion

Mesh-to-CAD. Table 1 reports quantitative mesh-to-CAD results on DeepCAD, Fusion360, and ABC, stratified by CAD complexity measured using STEP face count. Across all benchmarks, CADFit consistently outperforms prior methods in IoU, Chamfer Distance, and Invalid Ratio.

Learning-based baselines such as GenCAD-3D and Cadrille exhibit limited generalization beyond their training distributions. While these methods achieve reasonable performance on datasets similar to their training data, their reconstruction quality degrades sharply on more diverse benchmarks such as Fusion360 and on higher-complexity ABC subsets. This behavior is particularly evident in the Hard ABC split, where GenCAD-3D and Cadrille frequently oversimplify geometry or fail to recover a valid sequence.

In contrast, CADFit maintains strong performance across datasets and complexity levels. On the ABC Hard subset, CADFit more than doubles the median IoU of the strongest prior method (CAD-Recode) (Figure 6), demon-

strating its ability to recover long and interacting construction sequences. These gains highlight the importance of geometry-driven optimization and explicit program validation when reconstructing complex CAD models. Qualitative comparisons in Fig. 4 further show that CADFit more faithfully recovers thin structures, rotational primitives, and multi-step compositions that are often missed by purely learning-based approaches.

Image-to-CAD. Table 2 reports end-to-end image-to-CAD results on the ABC subsets. Despite noise and artifacts introduced during image-based reconstruction, the Hunyuan3D + preprocessing + CADFit pipeline achieves the best performance across all complexity levels. This result underscores the advantage of decomposing image-to-CAD reconstruction into a multi-stage process, where each component addresses a well-defined subproblem: perception models recover approximate surface geometry, while CADFit performs structured parametric reconstruction. Additional results and visualizations can be found in Appendix A.

Table 2. Image-to-CAD construction sequence reconstruction results on the ABC dataset subsets easy, medium and hard.

Method	ABC								
	Easy			Medium			Hard		
	IoU↑	CD↓	IR↓	IoU↑	CD↓	IR↓	IoU↑	CD↓	IR↓
GenCAD-3D	0.310	0.104	0.047	0.246	0.112	0.107	0.169	0.096	0.207
CAD-Coder	0.405	0.086	0.013	0.2912	0.103	0.027	0.1710	0.104	0.053
Cadriille	0.438	0.103	0.027	0.385	0.113	0.000	0.226	0.106	0.020
CAD-Recode	0.437	0.063	0.000	0.391	0.071	0.000	0.237	0.074	0.040
CADFit (Ours)	0.669	0.036	0.000	0.545	0.037	0.000	0.380	0.041	0.000
Improvement (%)	↑ 52.9	↓ 42.9	0.0	↑ 39.4	↓ 47.9	0.0	↑ 60.3	↓ 44.6	↓ 100.0

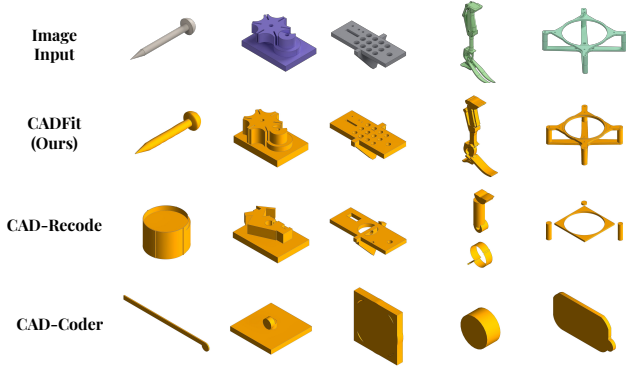


Figure 5. Qualitative image-to-CAD reconstruction results comparing CADFit with prior methods.

Construction Sequence Quality. Beyond geometric accuracy, CADFit recovers compact construction sequences. As shown in Table 3, in **95.1%** of cases CADFit achieves higher IoU than competing methods, and in **69.8%** of cases it simultaneously attains the highest IoU and the shortest valid sequence. Moreover, in **85%** of cases CADFit generates construction sequences that are shorter than the ground-truth DeepCAD programs while achieving **IoU > 0.9** (Figure 8).

Table 3. Breakdown of CADFit wins across IoU and construction sequence length against Cadriille and CAD-Recode on the complex DeepCAD subset (N = 300).

Outcome	Percentage
Wins both IoU and length	69.8%
Wins IoU only	25.3%
Wins length only	3.9%
Wins neither	1.1%

This behavior arises from CADFit’s optimization-driven formulation, which retains only operations that improve geometric fidelity rather than replicating potentially redundant or suboptimal construction histories. As a result, the recovered programs are easier to interpret and edit. Such programs may serve as improved training data for learning-

based CAD models or as structured exploration policies for reinforcement learning approaches to CAD design.

Contribution of the Learned Sketch Prior. CADFit uses learning only to filter candidate sketches, but this filter is essential for tractability. Initial extraction yields hundreds of plausible profiles, many noisy, fragmented, or irrelevant. Evaluating all of them with CAD-kernel calls is costly, so the no-prior baseline randomly samples 100 loops to keep optimization tractable. The learned prior replaces this random budget with a ranked subset of the most useful sketches.

Table 4. Effect of the learned sketch prior on quality and runtime across ABC splits. Under the same tractable candidate budget, the prior selects useful profiles instead of random loops, improving both scalability and reconstruction quality.

ABC subset	IoU change	Speedup	Time w/o prior (s)	Time w/ prior (s)
Easy	-0.0033	1.42×	303.0	213.3
Medium	+0.0032	1.62×	396.1	244.4
Hard	+0.2285	2.90×	903.7	311.8

The prior ranks profiles by their likelihood of appearing in the final CAD program. Table 4 shows speedups from 1.42×

 on ABC-easy to 2.90× on ABC-hard. On the hardest split, it also raises IoU by +0.2285 because the prior selects high-value sketches within the candidate budget, whereas the no-prior setting samples loops randomly. Thus, learning in CADFit acts as a guide rather than a generator: it preserves geometric optimization while replacing random search with a smaller, higher-quality candidate set.

Robustness to Noisy Meshes and Preprocessing. CADFit works best on clean, watertight meshes, since residual refinement uses boolean complements. Image-based reconstructions, however, are often noisy, dense, non-watertight, or topologically flawed. CADFit can still run its initial pass on such inputs, but full residual refinement may require preprocessing. Noise mainly increases cost rather than causing direct failure. It creates fragmented sketch candidates and slows IoU scoring. Simple preprocessing, such as Taubin smoothing and decimation, improves both speed and quality.

Table 5 shows this on Hunyuan3D meshes. Thus, CADFit is compatible with image-to-3D meshes, but preprocessing helps improve speed and robustness.

Table 5. Effect of lightweight preprocessing on image-based meshes.

Input setting	IoU	Runtime (s)
Noisy mesh	0.876	1255
Smoothed + decimated	0.914	456

Hyperparameter Selection. To justify the default configuration used in our main experiments, we ablate key hyperparameters controlling candidate generation, sketch extraction, residual refinement, and parameter search. The selected defaults achieve the best reconstruction quality among the tested settings while maintaining practical runtime. Overall, these results show that our configuration is chosen based on empirical reconstruction quality and efficiency. Full results are provided in Appendix H.



Figure 6. Photo-to-CAD reconstruction using Hunyuan3D and CADFit. Given realistic images of mechanical components, the pipeline reconstructs intermediate meshes and then recovers executable CAD construction sequences.

Application: Photo-to-CAD Reconstruction. We show another application of CADFit by using it for photo-to-CAD reconstruction on realistic images of mechanical components. We generate photo-style renderings from a component dataset (Kim et al., 2020) using Gemini (Team et al., 2023), reconstruct intermediate meshes with Hunyuan3D, and recover executable CAD construction sequences with CADFit. The results show that CADFit can recover structured programs beyond clean synthetic renderings, even when realistic inputs introduce mesh reconstruction artifacts. Qualitative examples are shown in Fig. 6.

5. Limitations

CADFit trades inference speed for validity and reconstruction quality. It is slower than feed-forward learned methods,

which typically run in seconds, since CADFit requires minutes for iterative optimization and CAD-kernel validation (Appendix A for timings). Its expressivity is also limited by the current operation set: extrusions, revolutions, fillets, and chamfers. This covers many engineering parts but can fail on shapes whose construction requires unsupported operations such as lofts, sweeps, or intersections. These unsupported operations also explain many of our observed failure cases. CADFit struggles with geometries dominated by smooth, high-curvature surfaces, since they often lack stable planar faces or slicing sections from which reliable sketches can be extracted (Appendix F). Such failures are not simply optimization errors; they reflect a mismatch between the target shape and the sketch-and-operation vocabulary currently supported by CADFit. CADFit further depends on the quality of the input mesh. Full residual refinement benefits from clean, watertight meshes because complement computation relies on robust boolean operations. For noisy, highly tessellated, or non-watertight meshes from image-based reconstruction models, lightweight preprocessing such as smoothing and decimation is often needed to improve robustness and runtime. Failures in upstream image-to-3D reconstruction can therefore propagate to the recovered CAD program.

6. Conclusion

We introduced **CADFit**, an optimization-based method for reconstructing structured, executable CAD construction sequences from geometric input. By formulating reconstruction as iterative fitting with explicit program validation, CADFit reliably recovers complex parametric operations while guaranteeing validity. Across mesh-to-CAD and image-to-CAD benchmarks, CADFit outperforms prior methods in geometric accuracy, robustness, and construction sequence quality, with especially large gains on high-complexity designs. CADFit adopts a decoupled design in which perception models recover surface geometry and parametric reconstruction is performed via geometry-driven optimization. This allows the same pipeline to operate across meshes and images and enables improvements in upstream reconstruction to transfer directly to CAD inference. Beyond precise geometric reconstruction, CADFit produces compact construction sequences that are often shorter than those in existing CAD datasets and prior methods. Future work includes extending the operator set to lofts, sweeps, pattern features and intersections, using CADFit to build higher-quality and complexity CAD datasets, and leveraging it as a structured policy for training learning-based or RL CAD agents with geometry feedback.

Impact Statement

This work aims to improve efficiency and structure in CAD reconstruction by recovering compact and precise construction programs from geometric observations. By enabling reusable and editable CAD representations, the proposed method may support faster engineering iteration and reduce reliance on manual modeling workflows. We do not foresee any immediate ethical or societal risks beyond those commonly associated with automation in engineering design tools.

Acknowledgments

This work was supported by a gift from Altair to the DeCoDE Lab.

References

- Alam, M. F. and Ahmed, F. Gencad: Image-conditioned computer-aided design generation with transformer-based contrastive representation and diffusion priors. *Transactions on Machine Learning Research*, 2024.
- Alrashedy, K., Tambwekar, P., Zaidi, Z. H., Langwasser, M., Xu, W., and Gombolay, M. Generating cad code with vision-language models for 3d designs. In *International Conference on Learning Representations*, volume 2025, pp. 52236–52262, 2025.
- Camba, J. D., Contero, M., and Company, P. Parametric cad modeling: An analysis of strategies for design reusability. *Computer-aided design*, 74:18–31, 2016.
- Contero, M., Pérez-López, D., Company, P., and Camba, J. D. A quantitative analysis of parametric cad model complexity and its relationship to perceived modeling complexity. *Advanced Engineering Informatics*, 56:101970, 2023.
- contributors, C. Cadquery, February 2026. URL <https://doi.org/10.5281/zenodo.18633916>.
- Doris, A. C., Alam, M. F., Heyrani Nobari, A., and Ahmed, F. Cad-coder: An open-source vision-language model for computer-aided design code generation. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89220, pp. V03AT03A031. American Society of Mechanical Engineers, 2025.
- Du, T., Inala, J. P., Pu, Y., Spielberg, A., Schulz, A., Rus, D., Solar-Lezama, A., and Matusik, W. Inversecsg: Automatic conversion of 3d models to csg trees. *ACM Transactions on Graphics (TOG)*, 37(6):1–16, 2018.
- Garland, M. and Heckbert, P. S. Surface simplification using quadric error metrics. In *Proceedings of the 24th annual conference on Computer graphics and interactive techniques*, pp. 209–216, 1997.
- Jayaraman, P. K., Lambourne, J. G., Desai, N., Willis, K., Sanghi, A., and Morris, N. J. Solidgen: An autoregressive model for direct b-rep synthesis. *Transactions on Machine Learning Research*, 2022.
- Kania, K., Zieba, M., and Kajdanowicz, T. Ucsq-net-unsupervised discovering of constructive solid geometry tree. *Advances in neural information processing systems*, 33:8776–8786, 2020.
- Karadeniz, A. S., Mallis, D., Rukhovich, D., Cherenkova, K., Kacem, A., and Aouada, D. MiCADangelo: Fine-grained reconstruction of constrained CAD models from 3d scans. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=GoFYIRUVaz>.
- Khan, M. S., Dupont, E., Ali, S. A., Cherenkova, K., Kacem, A., and Aouada, D. Cad-signet: Cad language inference from point clouds using layer-wise sketch instance guided attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4713–4722, 2024.
- Kim, S., Chi, H.-g., Hu, X., Huang, Q., and Ramani, K. A large-scale annotated mechanical components benchmark for classification and retrieval tasks with deep neural networks. In *European conference on computer vision*, pp. 175–191. Springer, 2020.
- Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., and Panozzo, D. Abc: A big cad model dataset for geometric deep learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 9601–9611, 2019.
- Kolodiazhnyi, M., Tarasov, D., Zhemchuzhnikov, D., Nikulin, A., Zisman, I., Vorontsova, A., Konushin, A., Kurenkov, V., and Rukhovich, D. cadrille: Multi-modal cad reconstruction with online reinforcement learning. *arXiv preprint arXiv:2505.22914*, 2025.
- Lambourne, J. G., Willis, K., Jayaraman, P. K., Zhang, L., Sanghi, A., and Malekshan, K. R. Reconstructing editable prismatic cad from rounded voxel models. In *SIGGRAPH Asia 2022 conference papers*, pp. 1–9, 2022.
- Lee, M., Zhang, D., Jambon, C., and Kim, Y. M. Brepdiff: Single-stage b-rep diffusion model. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, pp. 1–11, 2025.

- Li, M., Langbein, F. C., and Martin, R. R. Detecting design intent in approximate cad models using symmetry. *Computer-Aided Design*, 42(3):183–201, 2010.
- Li, P., Guo, J., Zhang, X., and Yan, D.-M. Secad-net: Self-supervised cad reconstruction by learning sketch-extrude operations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16816–16826, 2023.
- Li, Y., Lin, C., Liu, Y., Long, X., Zhang, C., Wang, N., Li, X., Wang, W., and Guo, X. Caddreamer: Cad object generation from single-view images. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 21448–21457, 2025.
- Liu, Y., Obukhov, A., Wegner, J. D., and Schindler, K. Point2cad: Reverse engineering cad models from 3d point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3763–3772, 2024.
- Man, K. Y. B., Nehme, G., Alam, M. F., and Ahmed, F. Videocad: A dataset and model for learning long-horizon 3d cad ui interactions from video. *Advances in Neural Information Processing Systems*, 38, 2026.
- Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., Fernandez, P., Haziza, D., Massa, F., El Nouby, A., et al. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research Journal*, 2024.
- Ren, D., Zheng, J., Cai, J., Li, J., and Zhang, J. Extrudenet: Unsupervised inverse sketch-and-extrude for shape parsing. In *European Conference on Computer Vision*, pp. 482–498. Springer, 2022.
- Rukhovich, D., Dupont, E., Mallis, D., Cherenkova, K., Kacem, A., and Aouada, D. Cad-recode: Reverse engineering cad code from point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9801–9811, 2025.
- Taubin, G. Curve and surface smoothing without shrinkage. In *Proceedings of IEEE international conference on computer vision*, pp. 852–857. IEEE, 1995.
- Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., Millican, K., et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Uy, M. A., Chang, Y.-Y., Sung, M., Goel, P., Lambourne, J. G., Birdal, T., and Guibas, L. J. Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11850–11860, 2022.
- Wang, N., Zhang, Y., Li, Z., Fu, Y., Liu, W., and Jiang, Y.-G. Pixel2mesh: Generating 3d mesh models from single rgb images. In *European Conference on Computer Vision (ECCV)*, 2018.
- Wang, S., Chen, C., Le, X., Xu, Q., Xu, L., Zhang, Y., and Yang, J. Cad-gpt: Synthesising cad construction sequence with spatial reasoning-enhanced multimodal llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 7880–7888, 2025.
- Willis, K. D., Pu, Y., Luo, J., Chu, H., Du, T., Lambourne, J. G., Solar-Lezama, A., and Matusik, W. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM Transactions on Graphics (TOG)*, 40(4):1–24, 2021.
- Wu, R., Xiao, C., and Zheng, C. Deepcad: A deep generative network for computer-aided design models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- Xiang, J., Lv, Z., Xu, S., Deng, Y., Wang, R., Zhang, B., Chen, D., Tong, X., and Yang, J. Structured 3d latents for scalable and versatile 3d generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 21469–21480, 2025.
- Xu, X., Willis, K. D., Lambourne, J. G., Cheng, C.-Y., Jayaraman, P. K., and Furukawa, Y. Skexgen: Autoregressive generation of cad construction sequences with disentangled codebooks. In *International Conference on Machine Learning*, pp. 24698–24724. PMLR, 2022.
- Xu, X., Jayaraman, P. K., Lambourne, J. G., Willis, K. D., and Furukawa, Y. Hierarchical neural coding for controllable cad model generation. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 38443–38461, 2023.
- Xu, X., Lambourne, J., Jayaraman, P., Wang, Z., Willis, K., and Furukawa, Y. Brep-gen: A b-rep generative diffusion model with structured latent geometry. *ACM Transactions on Graphics (TOG)*, 43(4):1–14, 2024.
- Yu, N., Alam, M. F., Hart, A. J., and Ahmed, F. Gencad-3d: Cad program generation using multimodal latent space alignment and synthetic dataset balancing. In *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, volume 89220, pp. V03AT03A015. American Society of Mechanical Engineers, 2025.
- Zhao, Z., Lai, Z., Lin, Q., Zhao, Y., Liu, H., Yang, S., Feng, Y., Yang, M., Zhang, S., Yang, X., et al. Hunyuan3d 2.0: Scaling diffusion models for high resolution textured 3d assets generation. *arXiv preprint arXiv:2501.12202*, 2025.

Table of Contents for Appendices

A. Additional Results and Visualizations	13
B. Metric Computation	15
C. Image-to-Mesh Preprocessing	16
D. Ablation Study: Trellis vs Hunyuan3D	16
E. Benchmark Construction	17
F. Failure Cases	18
G. Theoretical Properties of IoU-Guided Assembly	19
H. Hyperparameter Sensitivity and Robustness	21
I. Sketch Extraction and Profile Formation	22
J. Sketch Primitive Fitting	23
K. Geometry-Driven Operation Parameter Search	24
L. IoU-Guided Construction Sequence Search	24
M. Learning-Based Sketch Prior	25
N. Fillet and Chamfer Recovery	25

A. Additional Results and Visualizations

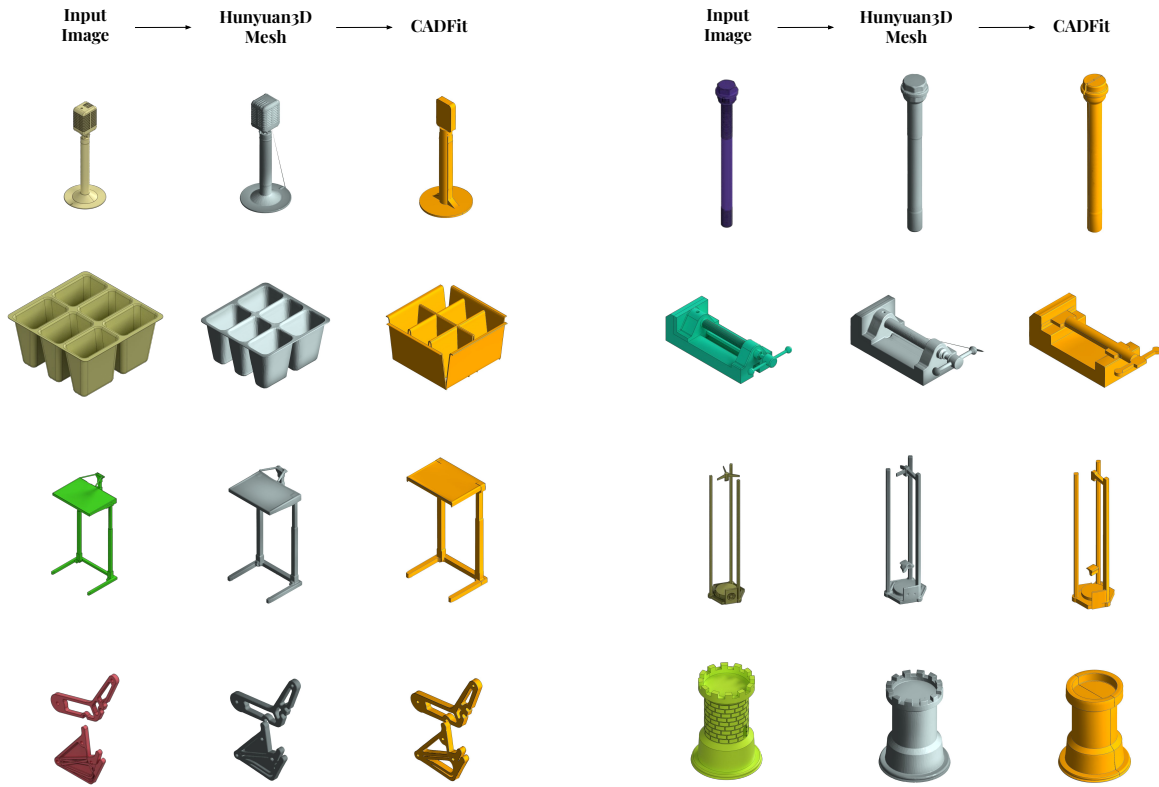


Figure 7. Qualitative Image-to-CAD results on the ABC Hard subset, showing the input image, intermediate Hunyuan3D mesh, and final CADFit reconstruction.

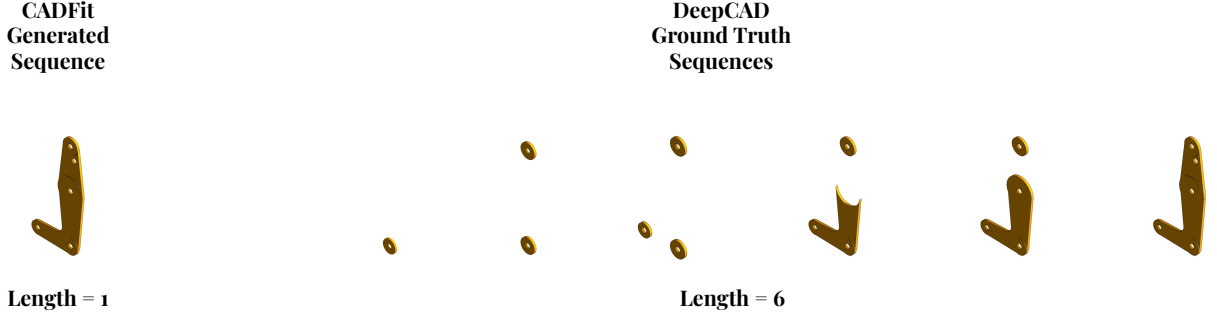


Figure 8. Example comparing CADFit output with DeepCAD ground-truth construction sequences. While DeepCAD programs may require multiple operations to form the target shape, CADFit frequently reconstructs the same geometry using fewer operations, yielding more concise CAD programs.

Table 6. Median mesh-to-CAD reconstruction results on DeepCAD, Fusion360, and ABC stratified by CAD complexity.

Method	DeepCAD		Fusion360		ABC					
	IoU↑	CD↓	IoU↑	CD↓	Easy		Medium		Hard	
					IoU↑	CD↓	IoU↑	CD↓	IoU↑	CD↓
GenCAD-3D (Yu et al., 2025)	0.225	0.129	0.163	0.143	0.232	0.136	0.177	0.130	0.104	0.118
CAD-Recode (Rukhovich et al., 2025)	0.900	0.018	0.874	0.017	0.885	0.019	0.627	0.029	0.267	0.043
Cadrille (Kolodiaznyi et al., 2025)	0.926	0.017	0.882	0.016	0.909	0.018	0.718	0.024	0.414	0.037
CADFit (Ours)	0.987	0.015	0.979	0.013	0.972	0.015	0.908	0.020	0.709	0.026

Table 7. Average runtime (seconds per shape) of CADFit across benchmarks.

Benchmark	Mean Runtime (s)
Mesh-to-CAD	
DeepCAD (complex subset)	242.3
Fusion360	235.0
ABC Easy	303.0
ABC Medium	396.1
ABC Hard	903.7
Image-to-CAD (ABC)	
ABC Easy (Hunyuan3D)	299.2
ABC Medium (Hunyuan3D)	343.5
ABC Hard (Hunyuan3D)	357.9

B. Metric Computation

B.1. Shape Alignment

Predicted CAD models may differ from ground-truth shapes by rigid transformations or global scale. To ensure that evaluation reflects geometric fidelity rather than coordinate conventions, we align predicted shapes to the ground truth prior to metric computation.

Given point clouds P_{gt} and P_{pred} sampled from the ground-truth and predicted surfaces, respectively, we estimate a similarity transform $(\mathbf{t}, \mathbf{R}, s)$ consisting of translation, rotation, and isotropic scale by minimizing symmetric Chamfer Distance:

$$\min_{\mathbf{t}, \mathbf{R}, s} \text{CD}(s \mathbf{R} P_{\text{pred}} + \mathbf{t}, P_{\text{gt}}).$$

Optimization is performed using derivative-free Powell search. The recovered transform is applied to the predicted mesh prior to computing CD and IoU.

B.2. Chamfer Distance

Chamfer Distance is computed between surface point clouds sampled uniformly from the predicted and ground-truth meshes. Specifically, given point sets P and $\hat{P}$, we compute the symmetric Chamfer Distance

$$\text{CD}(P, \hat{P}) = \frac{1}{2} \left(\frac{1}{|P|} \sum_{\mathbf{x} \in P} \min_{\mathbf{y} \in \hat{P}} \|\mathbf{x} - \mathbf{y}\|_2 + \frac{1}{|\hat{P}|} \sum_{\mathbf{y} \in \hat{P}} \min_{\mathbf{x} \in P} \|\mathbf{x} - \mathbf{y}\|_2 \right).$$

All ground-truth meshes are normalized to the box $[-1, 1]^3$, so reported CD values are directly comparable across shapes and datasets.

B.3. Volumetric IoU Computation

Volumetric IoU is computed directly using Boolean operations between the predicted and ground-truth solids. Given two watertight meshes or CAD solids A and B , we first compute their Boolean intersection and union:

$$A \cap B, \quad A \cup B.$$

The IoU is then defined as the ratio between the volume of the intersected geometry and the volume of the union geometry:

$$\text{IoU}(A, B) = \frac{\text{Vol}(A \cap B)}{\text{Vol}(A \cup B)}.$$

C. Image-to-Mesh Preprocessing

For image-to-CAD experiments, meshes reconstructed from RGB images may contain noise, holes, or non-manifold artifacts that violate CADFit’s geometric assumptions. We therefore apply a fixed preprocessing pipeline prior to CAD reconstruction.

Specifically, meshes produced by Hunyuan3D are post-processed as follows: (i) watertightness is enforced using surface repair; (ii) Taubin smoothing (Taubin, 1995) is applied to suppress high-frequency reconstruction noise while preserving overall shape; (iii) mesh decimation is performed to obtain a well-conditioned triangulation with bounded face count.

All preprocessing hyperparameters are fixed across datasets and complexity levels. A quantitative analysis of the impact of preprocessing on downstream CAD reconstruction is provided in this appendix.

D. Ablation Study: Comparing Trellis and Hunyuan3D on image-to-mesh

Table 8. Image-to-mesh reconstruction quality on the ABC dataset for Hunyuan3D and Trellis.

ABC Easy				
Method	IoU _{mean} ↑	IoU _{median} ↑	CD _{mean} ↓	CD _{median} ↓
Hunyuan3D	0.692	0.766	0.031	0.030
Trellis	0.202	0.103	0.043	0.030
ABC Medium				
Method	IoU _{mean} ↑	IoU _{median} ↑	CD _{mean} ↓	CD _{median} ↓
Hunyuan3D	0.602	0.680	0.031	0.026
Trellis	0.189	0.152	0.050	0.040
ABC Hard				
Method	IoU _{mean} ↑	IoU _{median} ↑	CD _{mean} ↓	CD _{median} ↓
Hunyuan3D	0.426	0.443	0.032	0.029
Trellis	0.187	0.147	0.055	0.045

E. Benchmark Construction

We construct evaluation benchmarks spanning a wide range of CAD complexity. Complexity is measured using STEP face count, which correlates strongly with construction sequence depth and operation diversity.

For ABC, shapes are stratified into Easy, Medium, and Hard subsets based on face count thresholds, and samples are drawn uniformly over linear face count within each bin. For DeepCAD and Fusion360, we sample from official test splits and publicly available galleries, respectively, following the protocol described in Section 4.

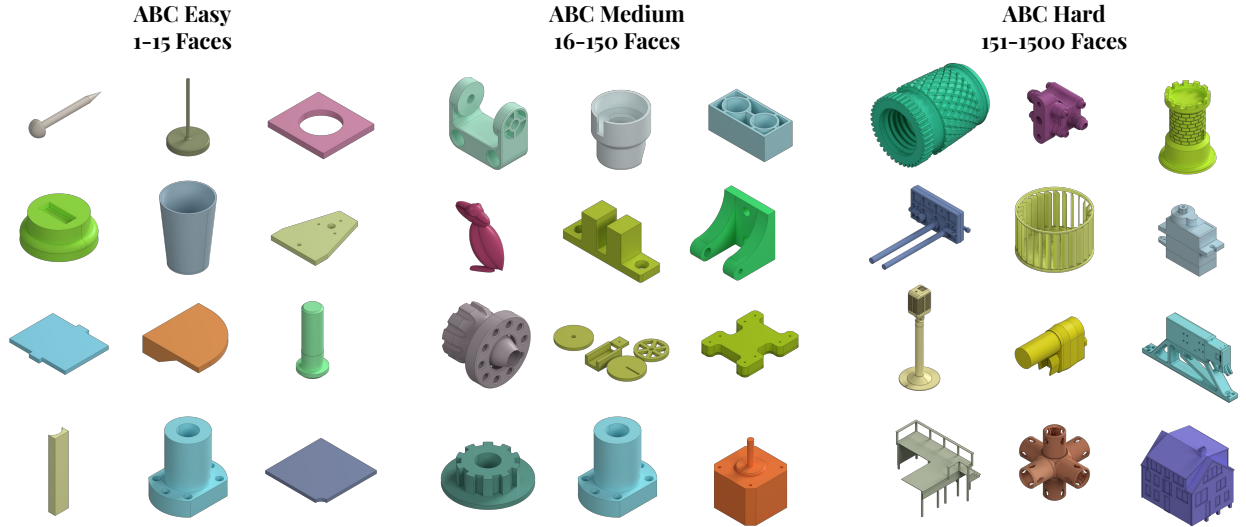


Figure 9. Qualitative visualizations of ABC CAD models stratified by complexity, rendered using Blender

F. Failure Cases



Figure 10. Failure cases of CADFit caused by unsupported operations and curved geometries.

Figure 10 illustrates the main characteristic failure modes of CADFit. Some failures arise from shapes whose construction relies on operations not currently supported by our kernel, such as lofts or sweeps. Despite this limitation, CADFit often recovers close geometric approximations using combinations of supported primitives.

More challenging cases occur when the ground-truth geometry is dominated by smoothly curved surfaces, for which no stable sketch can be extracted from planar faces or slicing sections. In such scenarios, the method struggles to recover precise constructions. A promising direction to address these cases is to project the shape onto candidate planes and use the resulting contours as sketches, which could significantly extend coverage to curved geometries.

G. Theoretical Properties of IoU-Guided Program Assembly with UNION/CUT

This section provides a formal explanation for why CADFit’s IoU-guided residual assembly (i) monotonically improves geometric fidelity (in symmetric-difference error / IoU) up to modeling and candidate-resolution limits, and (ii) produces compact, irredundant programs via backward IoU pruning, under standard feature-based CAD assumptions.

G.1. Setup

Let $\mathcal{M} \subset \mathbb{R}^3$ be the target solid and let S denote the current reconstructed solid. Define uncovered and overshoot residuals

$$R^+(S) = \mathcal{M} \setminus S, \quad R^-(S) = S \setminus \mathcal{M}.$$

Define the normalized symmetric-difference error

$$E(S) = \frac{\text{Vol}(\mathcal{M} \triangle S)}{\text{Vol}(\mathcal{M})} = \frac{\text{Vol}(R^+(S)) + \text{Vol}(R^-(S))}{\text{Vol}(\mathcal{M})}. \quad (2)$$

Let

$$a(S) = \frac{\text{Vol}(R^+(S))}{\text{Vol}(\mathcal{M})}, \quad b(S) = \frac{\text{Vol}(R^-(S))}{\text{Vol}(\mathcal{M})},$$

so that $E(S) = a(S) + b(S)$. IoU admits the identity

$$\text{IoU}(S, \mathcal{M}) = \frac{\text{Vol}(S \cap \mathcal{M})}{\text{Vol}(S \cup \mathcal{M})} = \frac{1 - a(S)}{1 + b(S)}. \quad (3)$$

Moreover, for all $a, b \geq 0$ with $a \leq 1$,

$$1 - \text{IoU}(S, \mathcal{M}) = 1 - \frac{1 - a}{1 + b} = \frac{a + b}{1 + b} \leq a + b = E(S). \quad (4)$$

Thus, decreasing $E(S)$ (symmetric-difference error) implies increasing IoU up to a monotone rescaling.

G.2. Assumptions

Assumption A1 (Feature-based realizability up to a floor). There exists a feature-based CAD program whose supported operations (sketch-based EXTRUDE/REVOLVE combined with Boolean UNION/CUT and optional FILLET/CHAMFER) can approximate $\mathcal{M}$ up to a modeling error floor $\varepsilon_{\text{model}} \geq 0$:

$$\inf_{S \in \mathcal{S}_{\text{CADFit}}} \frac{\text{Vol}(\mathcal{M} \triangle S)}{\text{Vol}(\mathcal{M})} \leq \varepsilon_{\text{model}}, \quad (5)$$

where $\mathcal{S}_{\text{CADFit}}$ denotes solids expressible by the operation set considered.

Assumption A2 (Residual-aligned sketch availability). When run on the current residual geometry, the sketch extraction procedure proposes at least one plane/section whose induced candidate operations have nontrivial overlap with any residual region of sufficient volume. In particular, candidate generation is not required to be optimal, only to propose sketches that can “touch” the residual to enable improvement.

Assumption A3 (Existence of a residual-improving candidate). At iteration t , for the current solid S_t , the candidate generator produces at least one update $(c_t, \circ_t)$ with $\circ_t \in \{\cup, \setminus\}$ such that

$$E(S_t \circ_t c_t) \leq \rho E(S_t) + \varepsilon_{\text{cand}}, \quad 0 < \rho < 1, \varepsilon_{\text{cand}} \geq 0. \quad (6)$$

Here $\varepsilon_{\text{cand}}$ captures candidate-resolution limits (e.g., finite parameter sweeps) and ρ captures a constant-factor progress condition.

Assumption A4 (IoU-monotone acceptance). The update executed by CADFit at each iteration is chosen to have non-decreasing IoU, i.e.,

$$\text{IoU}(S_{t+1}, \mathcal{M}) \geq \text{IoU}(S_t, \mathcal{M}),$$

equivalently (by (3)), it does not increase $E(\cdot)$ beyond monotone rescaling.

G.3. Geometric Refinement with Overshoot Correction via CUT

Theorem G.1 (Geometric refinement under IoU-guided residual assembly). *Under Assumptions A1–A4, the residual procedure produces solids $\{S_t\}_{t=0}^T$ satisfying*

$$E(S_T) \leq \rho^T E(S_0) + \frac{\varepsilon_{\text{cand}}}{1 - \rho}, \quad (7)$$

and consequently

$$\text{IoU}(S_T, \mathcal{M}) \geq 1 - \rho^T E(S_0) - \frac{\varepsilon_{\text{cand}}}{1 - \rho}. \quad (8)$$

Proof. By Assumption A3, there exists at iteration t a candidate update $(c_t, \circ_t)$ such that

$$E(S_t \circ_t c_t) \leq \rho E(S_t) + \varepsilon_{\text{cand}}.$$

By Assumption A4, CADFit selects an update with non-decreasing IoU, i.e., it does not worsen $E(\cdot)$ in the sense of the monotone relationship between IoU and E given by (3). Therefore, the executed update also satisfies

$$E(S_{t+1}) \leq \rho E(S_t) + \varepsilon_{\text{cand}}.$$

Unrolling the recurrence gives

$$E(S_T) \leq \rho^T E(S_0) + \varepsilon_{\text{cand}} \sum_{k=0}^{T-1} \rho^k = \rho^T E(S_0) + \varepsilon_{\text{cand}} \frac{1 - \rho^T}{1 - \rho} \leq \rho^T E(S_0) + \frac{\varepsilon_{\text{cand}}}{1 - \rho},$$

which proves (7). Finally, applying (4) yields (8). $\square$

Why CUT corrects overshoot. Suppose a UNION update adds extraneous volume $X \subset (c \setminus \mathcal{M})$. Then $X \subset R^-(S)$ by definition, i.e., it becomes overshoot residual. A subsequent CUT update $S \leftarrow S \setminus c'$ with $X \subseteq c'$ removes it and decreases $b(S)$, illustrating that overshoot introduced by unions is correctable through cuts, consistent with the residual formulation.

G.4. Compactness via Backward IoU Pruning

Let Π denote the assembled program (sequence/multiset of operations, including whether each operation is applied as UNION or CUT), and let $\text{Solid}(\Pi)$ denote the CAD-kernel solid obtained by executing Π . Backward IoU pruning repeatedly removes an operation whose removal yields the largest IoU gain (or smallest IoU drop), and terminates when any removal would decrease IoU.

Lemma G.2 (Irredundancy of the pruned program). *Let $\hat{\Pi}$ be the program returned by backward IoU pruning and $\hat{S} = \text{Solid}(\hat{\Pi})$. Then for every operation $o \in \hat{\Pi}$,*

$$\text{IoU}(\text{Solid}(\hat{\Pi} \setminus \{o\}), \mathcal{M}) < \text{IoU}(\hat{S}, \mathcal{M}),$$

i.e., removing any single operation strictly decreases IoU.

Proof. Backward pruning terminates only when no operation can be removed without decreasing IoU. Hence, at termination, every $o \in \hat{\Pi}$ satisfies the strict inequality above. $\square$

Lemma G.3 (Minimality among IoU-equivalent subsequences of the assembled program). *Let $\hat{\Pi}$ be the result of backward pruning applied to an initial assembled program Π_{init} . There is no strict subsequence $\Pi' \subset \hat{\Pi}$ such that*

$$\text{IoU}(\text{Solid}(\Pi'), \mathcal{M}) = \text{IoU}(\text{Solid}(\hat{\Pi}), \mathcal{M}).$$

Proof. Assume for contradiction that such Π' exists. Then some operation $o \in \hat{\Pi} \setminus \Pi'$ can be removed from $\hat{\Pi}$ without changing IoU, contradicting the termination condition of backward pruning (Lemma G.2). $\square$

Discussion. Theorem G.1 formalizes progressive refinement: under a standard residual progress condition (A3), IoU-guided residual updates reduce symmetric-difference error geometrically down to a floor determined by modeling/candidate limits. Lemmas G.2–G.3 formalize compactness: backward IoU pruning removes all operations that do not contribute to the final IoU, producing an irredundant program that is minimal among IoU-equivalent subsequences of the assembled program.

H. Hyperparameter Sensitivity and Robustness

We evaluate the sensitivity of CADFit to several key hyperparameters that affect sketch extraction, candidate generation, residual refinement, and parameter search. These ablations are conducted on held-out samples that are not included in the final test set, and are intended to justify the default configuration used throughout the main experiments.

Table 9 reports reconstruction quality, measured by mean IoU, and runtime, measured as mean reconstruction time per shape. The default configuration achieves the highest IoU among the tested settings. Most nearby hyperparameter changes lead to moderate degradation rather than complete failure, suggesting that CADFit is not overly sensitive to small changes in individual parameters. Larger drops identify important design choices. In particular, using both axis-aligned slicing and planar face-based sketch extraction is important: restricting the sketch source to axis-aligned slices or planar faces alone substantially reduces IoU. Similarly, increasing the number of slices does not necessarily improve performance, since it can introduce many noisy or redundant sketch candidates that increase runtime and make pruning more difficult.

The Chamfer Distance threshold and translation step size show the expected quality–runtime trade-off. A smaller translation step gives similar IoU but increases runtime, while a larger step reduces runtime at the cost of reconstruction quality. The residual threshold also affects this trade-off: a looser threshold terminates earlier and reduces runtime, while a stricter threshold increases computation without improving quality in this setting. Overall, these results support the default configuration as a good balance between accuracy and runtime.

Table 9. Hyperparameter sensitivity of CADFit on held-out samples. We report mean reconstruction IoU and mean runtime per shape. The default configuration, highlighted in bold, provides the best overall reconstruction quality while maintaining a reasonable runtime.

Hyperparameter	Value	Mean IoU $\uparrow$	Mean Time (s) $\downarrow$
CD threshold	0.01 (base)	0.826	330.6
	0.005	0.594	271.8
	0.05	0.791	398.6
δ sketch extraction tolerance	0.05 (base)	0.826	330.6
	0.01	0.764	302.9
	0.10	0.744	314.2
n_{slices} axis-aligned slicing	5 (base)	0.826	330.6
	10	0.510	393.0
	3	0.764	242.1
Residual threshold	0.02 (base)	0.826	330.6
	0.01	0.752	340.6
	0.05	0.764	300.1
Sketch source	axis + planar (base)	0.826	330.6
	axis only	0.668	202.9
	planar only	0.553	130.2
Translation step	0.01 (base)	0.826	330.6
	0.005	0.821	406.1
	0.02	0.684	257.4

I. Sketch Extraction and Profile Formation

This appendix details the geometric procedures used to extract sketch planes, section loops, and sketch profiles from the input mesh $\mathcal{M}$. All steps are deterministic and independent of learning.

Algorithm 2 Sketch Plane Extraction and Profile Grouping

Input: mesh $\mathcal{M}$
Output: sketch profile set $\mathcal{G}$
 Detect planar face clusters and axis-aligned slicing planes
for all sketch planes $(\mathbf{o}, \mathbf{n})$ **do**
 Intersect $\mathcal{M}$ with offset planes $\mathbf{o} \pm \delta \mathbf{n}$
 Extract closed section loops and project to plane
 Remove degenerate loops
 Group loops by polygon containment to form profiles
end for
return $\mathcal{G}$

I.1. Sketch Plane Extraction

Given a mesh $\mathcal{M}$, we construct a finite set of candidate sketch planes using two complementary strategies.

Planar face-based planes. We cluster triangle normals using angular similarity and retain clusters whose total surface area exceeds a threshold. Each cluster defines a plane with origin $\mathbf{o}_k$ (centroid of the cluster) and normal $\mathbf{n}_k$. To robustly capture boundaries, we slice $\mathcal{M}$ using planes at offsets $\mathbf{o}_k \pm \delta \mathbf{n}_k$.

Algorithm 3 Planar Sketch Extraction

Input: mesh M
Output: planar sketch set $\mathcal{S}_{\text{planar}}$
 Detect planar face clusters $\{(\mathbf{o}_k, \mathbf{n}_k)\}_{k=1}^K$ in M
for $k = 1$ **to** K **do**
 Define slicing planes at offsets $\mathbf{o}_k \pm \epsilon \mathbf{n}_k$
 Intersect M with slicing planes to obtain cross-sections
 Extract closed boundary loops $\mathcal{L}_k$
 if $\mathcal{L}_k \neq \emptyset$ **then**
 Define sketch $s_k \leftarrow (\mathbf{o}_k, \mathbf{n}_k, \mathbf{u}_k, \mathcal{L}_k)$
 Add s_k to $\mathcal{S}_{\text{planar}}$
 end if
end for
return $\mathcal{S}_{\text{planar}}$

Axis-aligned planes. To capture features not aligned with detected planar faces, we additionally introduce axis-aligned slicing planes. For each axis $a \in \{x, y, z\}$, we select a fixed number of slice positions $\{q_i\}$ corresponding to quantiles of the mesh extent along a , and define slicing planes ($a = q_i$).

Algorithm 4 Axis-Aligned Sketch Extraction

Input: mesh M
Output: axis-aligned sketch set $\mathcal{S}_{\text{axis}}$
for all axes $a \in \{x, y, z\}$ **do**
 Select slice positions $\{q_i\}$ as quantiles of M along axis a
 for all slice positions q_i **do**
 Slice M with plane ($a = q_i$)
 Extract closed loops $\mathcal{L}_{i,a}$
 Remove degenerate or rectangular loops
 if $\mathcal{L}_{i,a} \neq \emptyset$ **then**
 Define sketch $s_{i,a}$
 Add $s_{i,a}$ to $\mathcal{S}_{\text{axis}}$
 end if
 end for
end for
return $\mathcal{S}_{\text{axis}}$

I.2. Loop Extraction and Projection

For each slicing plane $(\mathbf{o}, \mathbf{n})$, we intersect $\mathcal{M}$ with nearby offset planes and extract closed polygonal loops. Each loop is projected back onto the original sketch plane and represented in a local 2D coordinate system. Loops are resampled to a fixed number of ordered points and filtered using area and minimum-length thresholds to remove degenerate cases.

I.3. Profile Grouping by Containment

Extracted loops may correspond to outer boundaries and interior holes. We construct a polygon containment graph in the sketch plane and group each outer loop with its contained inner loops. Each resulting group defines a sketch profile suitable for CAD operations.

Algorithm 5 Loop Grouping by Containment

Input: sketch set $\mathcal{S}$
Output: profile group set $\mathcal{G}$
for all sketches $s \in \mathcal{S}$ **do**
 Project all loops in s to the sketch plane
 Construct polygon containment graph
 Identify outer loops and their contained inner loops
 for all outer loops **do**
 Form profile group g with associated inner loops
 Add g to $\mathcal{G}$
 end for
end for
return $\mathcal{G}$

J. Sketch Primitive Fitting

Each sketch profile consists of an ordered set of 2D points $L^{2D} = \{(x_i, y_i)\}_{i=1}^N$ lying in a sketch plane. We approximate L^{2D} using a compact sequence of parametric primitives to obtain an executable CAD sketch.

Given a point subset $\mathcal{P} \subset \mathbb{R}^2$, we evaluate candidate fits for: (i) line segments via least-squares fitting, (ii) circles and arcs via algebraic circle fitting with RMS error constraints, and (iii) splines as a fallback representation. Segments are greedily grown until fitting error exceeds a tolerance relative to the loop scale. Adjacent colinear primitives are merged to reduce fragmentation.

The output is a sequence of primitives $\mathcal{S}(L)$ that approximates the original loop within tolerance and preserves loop topology.

Pseudo-code for the full segmentation procedure is provided in Algorithm.

K. Geometry-Driven Operation Parameter Search

This appendix details the continuous parameter selection procedure used to generate discrete EXTRUDE and REVOLVE candidates from a sketch profile.

Algorithm 6 Geometry-Driven Candidate Generation

Input: sketch profile g , mesh surface points $Q(\mathcal{M})$
Output: candidate set $\mathcal{C}_g$
 Sweep extrusion heights and identify stable intervals
 Canonicalize signed intervals and add extrusion candidates
if profile g is symmetric **then**
 Evaluate full revolutions about candidate axes
 Add best revolve candidate
end if
return $\mathcal{C}_g$

K.1. One-Sided Chamfer Distance

Let $Q(\mathcal{M}) = \{\mathbf{y}_j\}$ denote a surface point cloud sampled from $\mathcal{M}$. For an operation $o(g, \theta)$ applied to sketch profile g with parameters θ , let $P(g, \theta) = \{\mathbf{x}_i\}$ denote points sampled from the induced surface. We define the one-sided Chamfer Distance:

$$\mathcal{D}(\theta) = \frac{1}{|P(g, \theta)|} \sum_{\mathbf{x} \in P(g, \theta)} \min_{\mathbf{y} \in Q(\mathcal{M})} \|\mathbf{x} - \mathbf{y}\|_2^2. \quad (9)$$

K.2. Extrusion Interval Selection

For EXTRUDE, the parameter $\theta = h$ denotes a signed extrusion height along the sketch normal. We evaluate the Chamfer objective $\mathcal{D}(h)$ on a uniformly sampled, bounded interval $h \in [h_{\min}, h_{\max}]$. To identify geometrically valid extrusion ranges, we perform a monotonic error sweep and detect sharp increases in $\mathcal{D}(h)$ using finite-difference slope estimates. Specifically, we mark a boundary at h^* when the discrete derivative

$$\frac{\mathcal{D}(h_{i+1}) - \mathcal{D}(h_i)}{h_{i+1} - h_i}$$

exceeds a fixed threshold, indicating the onset of geometric overshoot.

Contiguous regions preceding such boundaries with $\mathcal{D}(h) \leq \epsilon$ are treated as stable intervals and merged when adjacent. From these, we retain two canonical candidates: (i) the smallest stable interval containing $h = 0$ and (ii) the largest stable interval. If a stable interval spans $h^- < 0 < h^+$, we reparameterize the extrusion by translating the sketch plane origin by $h^- \mathbf{n}$ and using height $h^+ - h^-$.

K.3. Revolution Axis Selection

For REVOLVE, candidates are restricted to full 360° revolutions. We generate a finite set of axis hypotheses consisting of the principal axes of the loop point distribution and four additional axes rotated by 45° . For each axis a , we evaluate $\mathcal{D}(a)$ and select the axis minimizing the Chamfer objective.

L. IoU-Guided Construction Sequence Search

CADFit assembles generated operation candidates using a two-stage IoU-guided search procedure. First, it greedily selects candidates that improve IoU with the target mesh. Second, it applies backward pruning to remove redundant operations whose deletion does not reduce reconstruction quality. This produces a compact construction sequence while preserving geometric fidelity.

Algorithm 7 IoU-Guided Construction Sequence Search

Input: sketch profiles $\mathcal{G}$, mesh $\mathcal{M}$
Output: CAD program Π , solid S
 Generate candidates $\mathcal{C}$ {Appendix K}
 $\Pi \leftarrow \emptyset, S \leftarrow \emptyset$
repeat
 $c^* \leftarrow \arg \max_{c \in \mathcal{C} \setminus \Pi} [\text{IoU}(S \cup c, \mathcal{M}) - \text{IoU}(S, \mathcal{M})]$
 if $\text{IoU}(S \cup c^*, \mathcal{M}) > \text{IoU}(S, \mathcal{M})$ **then**
 $\Pi \leftarrow \Pi \cup \{c^*\}, S \leftarrow S \cup c^*$
 else
 break
 end if
until no candidate improves IoU
repeat
 $o^* \leftarrow \arg \max_{o \in \Pi} [\text{IoU}(S_o^-, \mathcal{M}) - \text{IoU}(S, \mathcal{M})]$
 where $S_o^- := \text{Solid}(\Pi \setminus \{o\})$
 if $\text{IoU}(S_{o^*}^-, \mathcal{M}) \geq \text{IoU}(S, \mathcal{M})$ **then**
 $\Pi \leftarrow \Pi \setminus \{o^*\}, S \leftarrow S_{o^*}^-$
 else
 break
 end if
until no removable operation improves IoU
return (Π, S)

M. Learning-Based Sketch Prior

To reduce the number of sketch profiles considered during optimization, we train a binary classifier that predicts whether a candidate profile is likely to be used by the recovered CAD program.

Training data. Given a recovered program Π^* , profiles used by operations in Π^* are labeled positive, and remaining extracted profiles are labeled negative. Labels are generated automatically from CADFit reconstructions, without human annotation. We build the labeled pool from 16,719 STLs and use approximately 10^5 labeled sketch profiles for training, with held-out test shapes split at the STL level.

Model. The classifier computes a probability $f(\mathcal{M}, g) \in [0, 1]$ from a joint representation of the target mesh and candidate loop. For each loop, we render multi-view images of the normalized mesh $\mathcal{M}$ with the loop g overlaid, including an isometric view and a face-on view aligned with the loop plane. These views are encoded with DINOv2 and concatenated with lightweight geometric loop features, such as area, perimeter, point count, bounding-box extent, plane parameters, and axis alignment. The resulting feature vector is passed to a 3-layer MLP with nonlinear activations and a sigmoid output. In our implementation, the MLP uses two hidden layers with GELU activations and dropout, followed by a final binary prediction layer.

Usage. At inference time, each sketch profile is scored by $p = f(\mathcal{M}, g)$. CADFit uses these scores to bias profile selection toward loops that are more likely to contribute to the final reconstruction, replacing random loop sampling with a more informative candidate budget.

N. Fillet and Chamfer Recovery

Given a reconstructed solid S , we identify candidate edges for secondary features. For each edge, we apply a small test FILLET or CHAMFER operation and evaluate the resulting IoU. If IoU improves, the feature parameter (radius or length) is refined via one-dimensional line search to maximize IoU with $\mathcal{M}$.